



Bachelorthesis

Personalisierung von Chatbots im
Unternehmenskontext auf Basis
konversationsbasierter Nutzerpräferenzen

Prüfer:

Prof. Dr. Yvonne Gorniak

Verfasser:

Fabio Mirabile

102247

Auf dem Anger 4

33014 Bad Driburg

Wirtschaftsinformatik

Business Process Management

Eingereicht am:

8. Juni 2026

Sperrvermerk

Die vorgelegte Arbeit mit dem Titel „Adaptive Entscheidungsfindung in lernenden Chatbots auf Grundlage konversationsbasierter Nutzerpräferenzen“ beinhaltet vertrauliche Informationen und Daten von Phoenix Contact GmbH & Co. KG und der mit diesem verbundenen Unternehmen. Diese Arbeit darf nur vom Erst- und Zweitgutachter sowie berechtigten Mitgliedern des Prüfungsausschusses eingesehen werden. Eine Vervielfältigung und Veröffentlichung der Arbeit ist auch auszugsweise nicht erlaubt. Dritten darf diese Arbeit nur mit der ausdrücklichen Genehmigung von Phoenix Contact GmbH & Co. KG zugänglich gemacht werden. Zur Überprüfung von Plagiatsvergehen darf die Arbeit oder Teile davon einer Untersuchungsinstanz in Form einer Plagiat-Software vorgelegt werden. Der Sperrvermerk ist somit im Fall einer Plagiatsprüfung nicht wirksam.

Inhaltsverzeichnis

Sperrvermerk	II
Abbildungsverzeichnis	V
Listingverzeichnis	VI
1 Einleitung	1
1.1 Problemstellung	1
1.2 Vorstellung des Praxisunternehmens	2
1.3 Zielsetzung	2
1.4 Vorgehensweise	3
2 Theoretische Grundlagen	4
2.1 Large Language Models	4
2.1.1 Definition und Hintergrund	4
2.1.2 Funktionsweise	4
2.1.3 Fähigkeiten und Limitierungen	6
2.1.4 Prompting und Kontextverarbeitung	7
2.1.5 Nutzung über APIs	8
2.2 Personalisierung von Chatbots	8
2.2.1 Personalisierung und Nutzerpräferenzen	8
2.2.2 Speichermechanismen	9
2.2.3 Retrieval Augmented generation	10
2.3 Forschungsansatz und Methodik	11
2.3.1 Design Science Research	11
2.3.2 Methodik	12
3 Stand der Forschung	14
3.1 Personalisierte LLMs	14
3.2 Chat Historie	14
3.3 RAG Ansätze	14
4 Identifikation des Problemraums	16
4.1 Ist-Analyse	16
4.2 Rahmenbedingungen	17

4.3 Ziele	18
5 Evaluation	20
Anhang	21
Quellenverzeichnis	24
Ehrenwörtliche Erklärung	25

Abbildungsverzeichnis

Abbildung 1: Typischer RAG Prozess	11
Abbildung 2: Design Science Research Method (DSRM) Prozess Modell . .	13
Abbildung 3: PIA Architektur	19
Abbildung 4: Aktuelle Schaltlogik des Funktionsbausteins	22
Abbildung 5: Ansicht im Anlagen Ordner	22
Abbildung 6: Ergebnisse des Zeitvergleiches	23

Listingverzeichnis

1 Einleitung

1.1 Problemstellung

Der Einsatz von Chatbots in Unternehmen hat in den vergangenen Jahren deutlich an Bedeutung gewonnen. Insbesondere durch die Fortschritte im Bereich leistungsfähiger Large Language Models (LLMs) haben sich Chatbots zu einem zentralen Bestandteil der digitalen Unternehmensinfrastruktur entwickelt. Sie können sowohl Kunden als auch Mitarbeitende in unterschiedlichen Anwendungsszenarien unterstützen, indem sie Informationen bereitstellen, Prozesse automatisieren oder Handlungsempfehlungen ableiten.

Auch das Unternehmen Phoenix Contact hat im Zuge dieser Entwicklungen einen Enterprise-Chatbot in den operativen Betrieb integriert. Dieser ist bereits in der Lage, Nutzeranfragen strukturiert zu verarbeiten und dabei Informationen aus internen Wissensbasen zu berücksichtigen. Trotz dieser Funktionalitäten zeigt sich in der Praxis, dass die Antworten des Chatbots überwiegend standardisiert erfolgen. Individuelle Präferenzen, die der Nutzer innerhalb von Konversationen äußert, werden dabei nicht sitzungsübergreifend von dem Chatbot beachtet.

Dieses Problem führt dazu, dass die Erwartungen an die Leistungsfähigkeit des Systems nicht vollständig erfüllt werden können. Gerade aus der Sicht des Kunden kann ein ineffizientes Chatbot-System negative Einflüsse auf die Geschäftsbeziehung oder die Wahrnehmung des Unternehmens haben. Gleichzeitig beeinträchtigt es die interne Produktivität der Mitarbeiter, da Präferenzen redundant geäußert werden müssen, wodurch der Aufwand in einzelnen Konversationen steigt.

Vor diesem Hintergrund wird deutlich, dass die langfristige Berücksichtigung von Nutzerpräferenzen sowohl aus Kunden- als auch aus Mitarbeiterperspektive von zentraler Bedeutung ist. Um dies zu gewährleisten, müssen geeignete Mechanismen entwickelt werden, die Nutzerpräferenzen automatisiert erfassen, strukturiert speichern und in zukünftige Interaktionen integrieren können.

1.2 Vorstellung des Praxisunternehmens

Phoenix Contact ist ein global tätiges Familienunternehmen, das sich mit dem strategischen Ziel zur Umsetzung der „All Electric Society“ in der Elektroindustrie positioniert. Die Produkte von Phoenix Contact bieten unterschiedliche Lösungen für die Bereiche Automatisierung, Vernetzung und Elektrifizierung. Diese Arbeit wurde in der Abteilung "Data Science & Engineering"(DSE) verfasst, die ein Teil der Muttergesellschaft Phoenix Contact GmbH & Co.KG ist.

Die Abteilung DSE setzt sich unter anderem für Lösungen im Bereich der Generativen Künstlichen Intelligenz ein. Diese Lösungen werden ausschließlich von Phoenix Contact selbst eingesetzt und verfolgen die Absicht verschiedenste Prozesse mithilfe Intelligenter Assistenten zu unterstützen. In diesem Rahmen sind in der Vergangenheit beispielsweise Projekte wie ein Enterprise-Chatbot, ein Coding Assitent und ein Techbot entstanden.

Der Enterprise Chatbot namens "Phoenix Contact Intelligent Assistant"(PIA) spielt für diese Arbeit eine Zentrale Rolle. Es handelt sich dabei um einen Chatbot, der mit Unternehmensdaten erweitert wurde und dadurch präzisere Antworten über Phoenix Contact bereitstellen kann. Da sich die Umwelt im Bereich der künstlichen Intelligenz stetig weiterentwickelt, neue KI Modelle veröffentlicht werden und die Erwartungen der Nutzer zunehmend steigen, ist es auch Zukünftig wichtig, dass vorhandene Projekte wie der Enterprise Chatbot von der Abteilung evaluiert und weiterentwickelt werden.

1.3 Zielsetzung

Daher befasst sich diese Arbeit mit der Personalisierung von Chatbots im Unternehmenskontext. Ziel der Arbeit ist die Entwicklung und Evaluierung eines Konzepts, das Nutzerpräferenzen und relevante Kontexte aus vergangenen Konversationen berücksichtigen kann, um die Personalisierung zukünftiger Interaktionen zu verbessern. Der Fokus liegt dabei insbesondere auf Speicher Strukturen, wie

Die Zentrale Forschungsfrage dieser Untersuchung lautet: Welchen Einfluss haben Episodic Memory und Preference Memory auf die Nutzererfahrung eines Enterprise-Chatbots?

Das Ziel dieser Arbeit ist es nicht, einen vollständig personalisierten Chatbot zu entwickeln, der direkt Produktiv genommen werden kann. Vielmehr steht die Evaluierung des Konzepts im Fokus, auf dessen Grundlage am Ende eine Konzeptempfehlung für die Abteilung getroffen werden soll.

1.4 Vorgehensweise

Im ersten Teil der Arbeit werden die theoretischen Grundlagen dargestellt. In diesem Teil der Arbeit soll vor allem ein grundlegendes Verständnis für wichtige Bestandteile moderner Chatbot Architekturen wie beispielsweise Agenten, Large Language Models und Vektordatenbanken geschaffen werden, sodass Design Entscheidungen hinsichtlich der Zielsetzung nachvollziehbar sind. Gleichzeitig wird genauer erläutert welche Arten von Nutzerpräferenzen im Rahmen dieser Arbeit berücksichtigt werden. Darauf aufbauend wird der aktuelle Stand der Forschung untersucht. Dabei liegt der Fokus insbesondere auf bestehenden Ansätzen, die für die Personalisierung von Chatbots existieren.

Im anschließenden Praxisteil wird der Bezug zur betrieblichen Problemstellung bei Phoenix Contact hergestellt. Dazu wird zunächst der aktuelle Entwicklungsstand des eingesetzten Chatbots analysiert, um bestehende Prozessabläufe sowie mögliche Schwachstellen zu identifizieren

2 Theoretische Grundlagen

2.1 Large Language Models

2.1.1 Definition und Hintergrund

Large Language Models (LLMs) bilden einen zentralen Fortschritt für die Bereiche der generativen künstlichen Intelligenz und des Natural Language Processings (NLP). Der Begriff Large Language Modell bedeutet übersetzt so viel wie großes Sprachmodell und beschreibt allgemein ein System, dass auf großen Mengen textueller Daten trainiert wird, um natürliche Sprache verarbeiten und generieren zu können.

Das heutige Verständnis von LLMs ist dabei eng mit der sogenannten Transformer-Architektur verbunden, die 2017 durch Vaswani et al. in der Arbeit Attention Is All You Need vorgestellt wurde und den Bereich des NLP revolutioniert hat. Dabei handelt es sich um eine Spezielle Art des Neuronalen Netzes, welches im Gegensatz zu vorherigen Ansätzen den Fokus vollständig auf Self-Attention Mechanismen setzt. Diese erlauben es dem Modell die Beziehung von einem Wort zu jedem anderen Wort im Kontext parallel zu erfassen, wodurch es lernt welche Wörter für das Verständnis eines bestimmten Wortes besonders wichtig sind. Dadurch können Kontextinformationen und semantische Beziehungen auch über längere Textsequenzen hinweg berücksichtigt werden, ohne dass diese verloren gehen.

Der Hintergrund dieser Arbeit war es die Limitationen von State Of The Art Ansätzen in der Sequenz- und Sprachmodellierung zu adressieren. Ursprünglich wurde die Transformer-Architektur für maschinelle Übersetzung und einfache Sprachmodellierungsaufgaben verwendet. Erst spätere Forschungsarbeiten zeigten, dass die Architektur durch eine Skalierung der Modellgröße, der Trainingsdaten und der verfügbaren Rechenleistung Fähigkeiten entwickeln kann, die weit über die ursprünglich vorgesehenen Anwendungsbereiche hinausgehen.

2.1.2 Funktionsweise

Da die vollständige technische Funktionsweise von Large Language Models (LLMs) für das Verständnis dieser Arbeit nicht erforderlich ist, wird in diesem Abschnitt bewusst auf eine detaillierte Betrachtung verzichtet. Da LLMs jedoch die Grundlage des in

dieser Arbeit entwickelten Konzepts bilden, ist ein grundlegendes Verständnis ihrer Funktionsweise notwendig.

Die Funktionsweise eines auf der Transformer-Architektur basierenden LLMs lässt sich vereinfacht anhand der folgenden zentralen Konzepte beschreiben:

- **Pre-Training:** In dieser Trainingsphase lernt das Modell anhand großer Textmengen statistische Zusammenhänge, sprachliche Strukturen sowie allgemeines Weltwissen. Ziel ist es, das nächste Token innerhalb einer Sequenz möglichst präzise vorherzusagen.
- **Fine-Tuning:** Nach dem Pre-Training wird das Modell auf spezifische Aufgaben oder Anwendungsbereiche angepasst. Hierbei werden die bereits erlernten Fähigkeiten durch zusätzliche, zielgerichtete Trainingsdaten verfeinert.
- **Tokenisierung:** Vor der Verarbeitung wird der Eingabetext in kleinere Einheiten, sogenannte Token, zerlegt. Diese können Wörter, Wortbestandteile oder Satzzeichen umfassen.
- **Embedding:** Die Token werden in numerische Vektoren umgewandelt, sodass ihre semantischen Eigenschaften für das Modell mathematisch verarbeitet werden können.
- **Transformer-Architektur:** Die grundlegende Modellarchitektur moderner LLMs, die durch den Attention-Mechanismus eine effiziente Verarbeitung langer Textsequenzen ermöglicht.
- **Inferenz:** Während der Nutzung verarbeitet das Modell die Eingabe und erzeugt schrittweise neue Token, indem es für jedes mögliche Folgetoken Wahrscheinlichkeiten berechnet und daraus eine Auswahl trifft.

Auf Basis dieser Mechanismen verarbeitet das Modell Eingabesequenzen, indem es die einzelnen Token unter Berücksichtigung ihres jeweiligen Kontextes analysiert. Mithilfe der während des Trainings erlernten sprachlichen Muster und Zusammenhänge berechnet das Modell Wahrscheinlichkeiten für mögliche Folgetoken. Anschließend wird ein geeignetes Token ausgewählt und an die bestehende Sequenz angefügt.

Dieser Vorgang wiederholt sich iterativ, bis die gewünschte Ausgabe vollständig generiert wurde. Die Qualität der erzeugten Antwort hängt dabei maßgeblich von den während

des Trainings erlernten Zusammenhängen sowie von den im Prompt bereitgestellten Kontextinformationen ab.

2.1.3 Fähigkeiten und Limitierungen

LLMs weisen unterschiedliche Fähigkeiten auf, die sie für eine Vielzahl von Anwendungen nutzbar machen. Zu den wichtigsten Fähigkeiten gehört das sogenannte In-Context Learning. Dabei können Modelle Aufgaben anhand von Anweisungen oder Beispielen innerhalb des Prompts bearbeiten, ohne dass eine zusätzliche Anpassung der Modellparameter erforderlich ist. Darüber hinaus sind LLMs in der Lage, zusammenhängende Texte zu generieren, Inhalte zusammenzufassen, Fragen zu beantworten sowie Texte zwischen verschiedenen Sprachen zu übersetzen.

Zudem können viele aktuelle LLMs strukturierte Ausgaben erzeugen. Dies ermöglicht die Ausgabe von Informationen in fest definierten Formaten wie JSON, XML oder Tabellenstrukturen und erleichtert dadurch die Integration in Softwareanwendungen und automatisierte Verarbeitungsketten.

Trotz dieser Fähigkeiten weisen LLMs grundlegende Einschränkungen auf. Es ist wichtig zu betonen, dass LLMs kein tatsächliches Verständnis von Sprache oder Wissen besitzen. Stattdessen basieren ihre Antworten auf der probabilistischen Vorhersage des jeweils wahrscheinlichsten nächsten Tokens innerhalb einer gegebenen Sequenz. Die erzeugten Ergebnisse hängen somit maßgeblich von den statistischen Mustern ab, die während des Trainings erlernt wurden.

Aus diesem Grund kann es vorkommen, dass LLMs inhaltlich falsche oder frei erfundene Informationen erzeugen. Dieses Phänomen wird als Halluzination bezeichnet und stellt eine zentrale Herausforderung beim Einsatz solcher Modelle dar. Dabei generiert das Modell Aussagen, die plausibel erscheinen, jedoch nicht den tatsächlichen Fakten entsprechen.

Darüber hinaus sind die Antworten eines LLMs nicht vollständig deterministisch, so dass identische Anfragen zu unterschiedlichen Ergebnissen führen können. Aus diesen Gründen sollten die von LLMs erzeugten Inhalte insbesondere in Anwendungsbereichen mit hohen Anforderungen an Korrektheit und Zuverlässigkeit kritisch geprüft werden.

2.1.4 Prompting und Kontextverarbeitung

Die Interaktion mit einem LLM erfolgt über sogenannte Prompts. Der Begriff Prompt bedeutet „Aufforderung“ oder „Eingabe“ und beschreibt die Informationen, die einem Modell zur Generierung einer Antwort bereitgestellt werden. Prompts können sowohl einfache Fragen als auch komplexe Anweisungen, Rollenbeschreibungen oder zusätzliche Kontextinformationen enthalten. Die Qualität und Struktur eines Prompts haben dabei einen erheblichen Einfluss auf die Qualität der generierten Antworten.

Im Kontext moderner LLMs wird häufig zwischen verschiedenen Arten von Prompts unterschieden. System-Prompts definieren grundlegende Verhaltensregeln oder Rollen des Modells, beispielsweise die Rolle eines Kundenservice-Chatbots. User-Prompts enthalten die eigentliche Eingabe des Nutzers. Zusätzlich können weitere Informationen wie Dokumente, Gesprächsverläufe oder anwendungsspezifische Daten bereitgestellt werden.

Die gezielte Gestaltung solcher Eingaben wird als Prompt Engineering bezeichnet. Ziel des Prompt Engineerings ist es, durch eine geeignete Formulierung und Strukturierung der Eingaben die Qualität, Konsistenz und Relevanz der generierten Antworten zu verbessern. Insbesondere im Unternehmenskontext spielt Prompt Engineering eine wichtige Rolle, da das Verhalten eines Chatbots gezielt an spezifische Anwendungsfälle angepasst werden kann.

Für die Verarbeitung von Prompts steht einem LLM nur ein begrenzter Kontext zur Verfügung. Dieser wird durch das sogenannte Kontextfenster beschrieben, das die maximale Anzahl an Tokens angibt, die ein Modell gleichzeitig verarbeiten kann. Alle Informationen innerhalb dieses Fensters können bei der Antwortgenerierung berücksichtigt werden. Wird die maximale Größe überschritten, müssen Teile des bisherigen Kontexts entfernt oder zusammengefasst werden.

Die Größe des Kontextfensters ist insbesondere für dialogbasierte Systeme relevant. Bei längeren Konversationen besteht die Gefahr, dass frühere Informationen nicht mehr berücksichtigt werden können. Dies stellt insbesondere für personalisierte Chatbots eine Herausforderung dar, da Nutzerpräferenzen und frühere Interaktionen häufig über einen längeren Zeitraum hinweg relevant bleiben.

2.1.5 Nutzung über APIs

In der Softwareentwicklung werden LLMs überwiegend nicht lokal betrieben sondern über Application Programming Interfaces (APIs) von externen Anbietern eingebunden. Bekannte Unternehmen, wie Anthropic, OpenAi, Google oder Meta, bieten heute Modelle mit unterschiedlichen Größen an, wodurch sie ohne großen Aufwand integriert werden können. Dieser Umstand eröffnet Unternehmen zahlreiche Möglichkeiten LLMs effizient im Betrieb einzusetzen.

2.2 Personalisierung von Chatbots

2.2.1 Personalisierung und Nutzerpräferenzen

Unter Personalisierung von Chatbots versteht man die Anpassung von Dialogen und Antworten an individuelle Eigenschaften, Präferenzen oder Verhaltensweisen eines Nutzers. Ziel der Personalisierung ist es, die Relevanz, Konsistenz und Nutzerzufriedenheit innerhalb einer Interaktion zu erhöhen. Im Gegensatz zu generischen Chatbots, die allen Nutzern identische Antworten liefern, berücksichtigen personalisierte Systeme nutzerspezifische Informationen bei der Antwortgenerierung.

Für die Personalisierung spielen Nutzerpräferenzen eine zentrale Rolle. Diese beschreiben individuelle Vorlieben, Interessen oder Anforderungen eines Nutzers und können auf unterschiedliche Weise erfasst werden. Grundsätzlich wird zwischen expliziten und impliziten Nutzerpräferenzen unterschieden.

Explizite Nutzerpräferenzen werden direkt vom Nutzer angegeben. Beispiele hierfür sind bevorzugte Spracheinstellungen, Kommunikationsstile oder ausdrücklich genannte Interessen. Die Erfassung erfolgt meist durch Formulare, Einstellungen oder direkte Angaben innerhalb einer Konversation.

Implizite Nutzerpräferenzen werden dagegen aus dem Verhalten eines Nutzers abgeleitet. Hierzu zählen beispielsweise häufig behandelte Themen, wiederkehrende Anfragen oder bestimmte Nutzungsmuster. Im Gegensatz zu expliziten Präferenzen müssen diese Informationen durch Analyse vergangener Interaktionen identifiziert werden.

Die Berücksichtigung solcher Präferenzen ermöglicht es Chatbots, Antworten stärker auf die individuellen Bedürfnisse eines Nutzers auszurichten und langfristig konsistentere Interaktionen zu erzeugen.

2.2.2 Speichermechanismen

Damit Nutzerpräferenzen und frühere Interaktionen für zukünftige Konversationen verfügbar bleiben, benötigen personalisierte Chatbots geeignete Speichermechanismen. In der Literatur wird häufig zwischen Kurzzeit- und Langzeitgedächtnis sowie zwischen episodischem und semantischem Gedächtnis unterschieden.

Das Kurzzeitgedächtnis (Short-Term Memory) umfasst Informationen, die lediglich während einer einzelnen Konversation verfügbar sind. In LLM-basierten Chatbots wird dieses Gedächtnis typischerweise durch das Kontextfenster realisiert. Sämtliche Nachrichten innerhalb des Kontextfensters können bei der Antwortgenerierung berücksichtigt werden. Nach Beendigung der Konversation oder bei Überschreitung der Kontextgrenze gehen diese Informationen jedoch verloren.

Das Langzeitgedächtnis (Long-Term Memory) dient hingegen der dauerhaften Speicherung nutzerbezogener Informationen über mehrere Interaktionen hinweg. Die Informationen werden außerhalb des Modells gespeichert und können bei Bedarf erneut abgerufen werden. Dadurch können Präferenzen und Wissen auch über längere Zeiträume hinweg berücksichtigt werden.

Innerhalb des Langzeitgedächtnisses wird häufig zwischen semantischem und episodischem Gedächtnis unterschieden. Das semantische Gedächtnis speichert allgemeines Wissen über einen Nutzer. Dazu gehören beispielsweise persönliche Präferenzen, Interessen oder dauerhaft gültige Eigenschaften. Die Informationen sind unabhängig vom Zeitpunkt ihrer Erfassung und beschreiben den Nutzer auf einer abstrakten Ebene.

Das episodische Gedächtnis speichert dagegen konkrete Ereignisse oder Interaktionen aus vergangenen Gesprächen. Hierzu zählen beispielsweise frühere Anfragen, Problemlösungen oder besondere Gesprächssituationen. Im Gegensatz zum semantischen Gedächtnis besitzen diese Informationen einen zeitlichen und kontextbezogenen Bezug.

Die Kombination aus Kurzzeit- und Langzeitgedächtnis ermöglicht es personalisierten Chatbots, sowohl aktuelle Gesprächsinhalte als auch langfristige Nutzerinformationen bei der Antwortgenerierung zu berücksichtigen.

2.2.3 Retrieval Augmented generation

LLMs haben die Einschränkung, dass ihre Informationen auf den Trainingsdaten basieren. Daher können sie aktuelles Wissen oder Domänenspezifisches Wissen in der Antwortgenerierung nicht gezielt einbinden und neigen daher zur Halluzination. Retrieval Augmented Generation (RAG) ist ein Framework, welches es dem Anwender ermöglicht LLMs mit externen Wissensbasen zu erweitern. Dabei werden zur Laufzeit der Prompt Erstellung zusätzliche Daten aus Datenbanken abgefragt und im Kontextfenster implementiert. RAG hilft dabei die Einschränkungen von LLMs zu überwinden und die Antwortgenerierung zu optimieren.

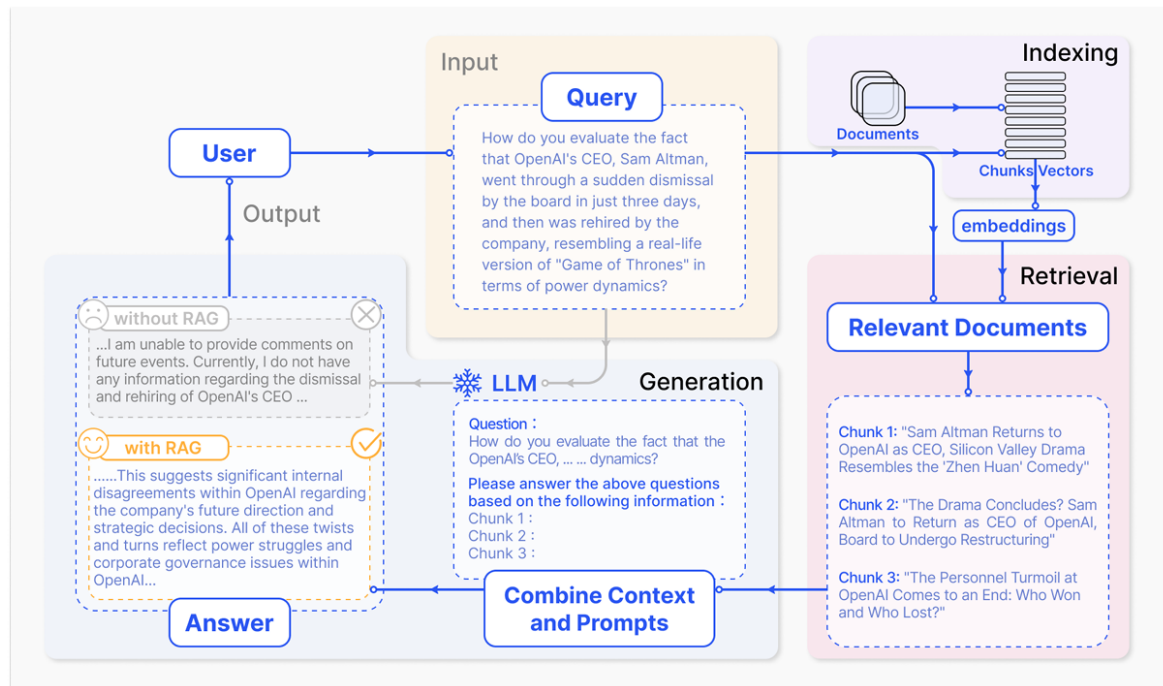
Ein typischer RAG Prozess kann der Abbildung 1 entnommen werden. Der Prozess startet mit dem Nutzer, der einen Prompt formuliert. Im Anschluss wird mithilfe des Prompts eine Query vorbereitet. Ein wichtiger Zwischenschritt in dem Prozess ist das Indexing. Beim Indexing werden Inhalte in sogenannte Chunks eingeteilt. Aus den Chunks werden mithilfe von Embedding Modellen Vektorrepräsentationen erzeugt. Das Embedding Modell sorgt dafür, dass aus semantisch ähnlichen Inhalten, ähnliche Vektorrepräsentationen erzeugt werden. Die Ähnlichkeit wird durch Metriken wie der euklidischen Distanz oder dem Skalarprodukt gemessen. Diese Vektoren werden Anschließend in Vektordatenbanken gespeichert.

Unter dem Begriff Vektordatenbank versteht man ein Datenbanksystem, welches die effiziente Speicherung, Indexierung und semantische Suche hochdimensionaler Vektoren unterstützt. Im Gegensatz zu klassischen relationalen Datenbanken sind Vektordatenbanken darauf spezialisiert semantische Suchverfahren nach inhaltlich ähnlichen Inhalten bereitzustellen. Die Suche basiert meist auf approximativen Verfahren zur Bestimmung der nächsten Nachbarn eines Query-Vektors, wodurch selbst in großen Datenmengen eine effiziente Suche gewährleistet werden kann.

Bevor der Prompt dem LLM zur Antwortgenerierung übergeben wird, wird die query geindext und es wird in der Datenbank nach ähnlichen Chunks gesucht, die für den Kontext relevant sein könnten. Diese werden anschließend mit dem Prompt kombiniert und

zusammen an das LLM übertragen. Dieser Prozess führt dazu, dass Fragen besser verarbeitet werden und zusätzliche Informationen bereitgestellt werden.

Abbildung 1: Typischer RAG Prozess



Quelle: Gao et al., 2023

2.3 Forschungsansatz und Methodik

2.3.1 Design Science Research

Der wissenschaftliche Forschungsansatz Design Science Research, stellt die Auseinandersetzung mit Artefakten in den Mittelpunkt und ist für diese Arbeit geeignet. Bei der Design Science Research geht es primär darum Probleme, die in einem bestimmten praxisbezogenen Kontext bestehen, strukturiert zu bearbeiten. Das Ziel ist es, eine Verbesserung in dem Problemkontext zu erreichen, indem sich der Forscher mit der Interaktion zwischen Problemkontext und dem Artefakt auseinandersetzt. Hierzu entwickelt der Forscher sogenannte Artefakte. Im Fokus steht dabei nicht das Gestalten völlig neuer Artefakte oder einer Invention. Vielmehr geht es bei der DSR darum das erstellte Artefakt am Ende der Durchführung hinsichtlich seiner Wirksamkeit in Bezug auf das adressierte Problem zu evaluieren.

Artefakte spielen im Bereich der DSR eine zentrale Rolle. Knut Linke beschreibt ein Artefakt als eine „[...] vom Menschen geschaffene Lösung für ein konkretes, praxisbezogenes Problem [...]“ (Linke, 2026) und erläutert, dass allein die Entwicklung und Weiterentwicklung von etwas bereits ein Artefakt darstellen kann. In der Praxis kann es sich dabei konkret um Modelle, Konzepte, Softwarearchitekturen oder Prototypen handeln. Allerdings besteht die Anforderung, dass das Artefakt evaluierbar sein muss und vom Forscher problemorientiert untersucht werden kann.

Der Prozess der DSR teilt sich in mehrere Aktivitäten auf, die bei der Durchführung beachtet werden müssen. Diese Arbeit orientiert sich an dem Modell nach Peffers, welches der Abbildung 2 entnommen werden kann.

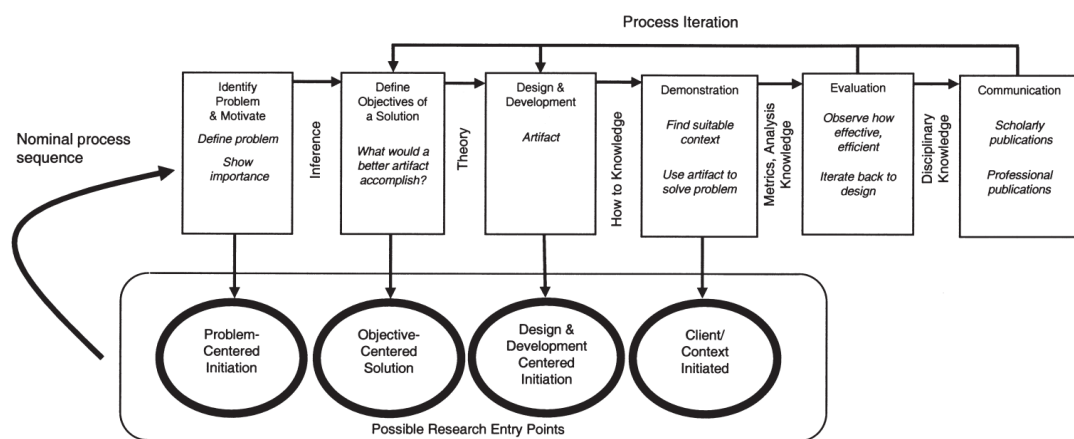
Der Prozess beginnt mit der Problem Identifikation und Motivation. Hier sollte der Forscher das Problem genau beschreiben und die Rahmenbedingungen für das Projekt erläutern. Dabei sollte sowohl aus Empirischer als auch aus Sicht der Praxis eine Relevanz für das Projekt begründet werden. In dieser Phase sollten vor allem Forschungsfragen festgelegt werden, die durch die Durchführung beantwortet werden sollen. Im Anschluss daran müssen aus der Problemstellung die konkreten Ziele abgeleitet werden.

Der nächste Schritt ist es das Artefakt zu erstellen und zu entwickeln. In dieser Phase werden wichtige Design Entscheidungen für das Artefakt getroffen.

Sobald das Artefakt erstellt wurde, muss demonstriert werden, dass es das Problem tatsächlich lösen kann. Es ist also ein Prove of Concept notwendig.

Im letzten Schritt wird die Lösung vor dem Hintergrund des Problemraums und der Zielsetzung evaluiert. Hierzu werden wissenschaftliche Methoden verwendet um die Validität der Lösung empirisch bewerten zu können. Sollte die Evaluation ergeben, dass das Artefakt das Problem und die Ziele nicht lösen kann, ist es notwendig zurück in die Design Phase zu wechseln und den Prozess erneut zu iterieren.

2.3.2 Methodik

Abbildung 2: Design Science Research Method (DSRM) Prozess Modell

3 Stand der Forschung

Die Personalisierung von Chatbots stellt gerade im Unternehmenskontext eine aktuelle Herausforderung dar. Ziel dieses Abschnittes ist es in diesem Zusammenhang aktuelle Techniken und Ansätze aus der Forschung vorzustellen, die sich ebenfalls mit dieser Fragestellung auseinandergesetzt haben.

3.1 Personalisierte LLMs

Ein oft behandeltes Thema bilden personalisierte LLMs. Dabei geht es darum LLMs explizit auf die Präferenzen eines Nutzers abzustimmen, sodass diese antworten generieren, die direkt auf eine Person zugeschnitten sind. In der Praxis stellt dies jedoch eine grundlegende Schwierigkeit für Unternehmen dar, da moderne LLMs meist in den Händen der Anbieter sind und sich nicht einfach so neu trainieren lassen. Aus diesem Grund bildet dieser Ansatz, sofern nicht eigene LLMs aufgebaut werden, einen unrealistischen Ansatz für diese Arbeit.

3.2 Chat Historie

Ein weiterer Ansatz der in vielen Arbeiten erwähnt wird, ist es die Chat Historie als Kontext in den Prompt zu integrieren. Das LLM könnte in diesem Zusammenhang selbst, die Präferenzen aus der Chat Historie extrahieren und anhand dieser eine Antwort generieren. In Anbetracht der Größe des Kontextfensters für LLMs, zeigen sich dabei jedoch grundlegende Herausforderungen, da die Anzahl an Tokens begrenzt ist.

3.3 RAG Ansätze

Im Jahr 2025 stellten Zhao et al. in ihrem wissenschaftlichen Paper "Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs" ihren Benchmark (PrefEval) zur Evaluation der Präferenzverarbeitung in konversationsbasierten

LLM-Systemen vor. In dieser Arbeit wurde untersucht, wie gut LLM Modelle Nutzerpräferenzen über längere Konversationen hinweg erkennen, speichern und in Antworten berücksichtigen können.

Die Ergebnisse des Benchmarks zeigen, dass moderne LLMs grundsätzlich in der Lage sind, Nutzerpräferenzen innerhalb von Konversationen zu berücksichtigen. Gleichzeitig zeigt sich jedoch, dass insbesondere die konsistente Verarbeitung von Präferenzen über längere und mehrere Konversationen hinweg weiterhin eine zentrale Herausforderung darstellt.

Die Ergebnisse des Benchmarks zeigen, dass die Fähigkeit von LLMs zur Berücksichtigung von Nutzerpräferenzen mit zunehmender Gesprächslänge deutlich abnimmt. Während Präferenzen in kurzen Konversationen häufig noch korrekt verarbeitet werden können, sinkt die Genauigkeit bei längeren Dialogen erheblich. Insbesondere Präferenzen, die früh innerhalb einer Konversation genannt wurden, werden von den Modellen häufig nicht mehr zuverlässig berücksichtigt.

Die Ergebnisse von PREFEVAL zeigen jedoch auch, dass insbesondere RAG-basierte Ansätze Halluzinationen reduzieren und die konsistente Berücksichtigung von Nutzerpräferenzen verbessern können. Durch den gezielten Abruf relevanter Präferenzinformationen aus externen Speicherstrukturen gelingt es den Modellen besser, frühere Nutzerangaben in späteren Konversationen erneut zu berücksichtigen. Dies ist insbesondere für personalisierte Chatbots im Unternehmenskontext relevant, da Nutzerpräferenzen häufig über längere Zeiträume hinweg konsistent verarbeitet werden müssen.

Externe Retrieval-Mechanismen bilden somit einen vielversprechenden Ansatz, um die Limitierungen kontextbasierter Präferenzverarbeitung moderner LLMs zu adressieren.

4 Identifikation des Problemraums

4.1 Ist-Analyse

Aus der zugrunde liegenden Problemstellung dieser Arbeit geht hervor, dass der betrachtete Chatbot bislang keine Funktionen zur Personalisierung von Nutzeranfragen bereitstellt. Dies führt zu zwei zentralen Einschränkungen des Systems:

- Die Unfähigkeit, vergangene Konversationen bei der Verarbeitung aktueller Anfragen zu berücksichtigen
- Die fehlende Erfassung sowie Nutzung individueller Nutzerpräferenzen

Für die Entwicklung eines geeigneten Artefakts ist ein vertieftes Verständnis des Problemraums wichtig, damit technische Schwachstellen und Defizite behoben werden können. Daher ist es sinnvoll den aktuellen Stand des behandelten Chatbots näher zu untersuchen und zu analysieren. Zur Veranschaulichung dient die Abbildung 3.

Insgesamt unterteilt sich die Architektur in eine Präsentations- und Orchestrierungsschicht. Die Präsentationsschicht basiert hauptsächlich auf einem React Frontend, worüber Authentifizierte Nutzer die Möglichkeit haben eigene Chats zu starten und Anfragen zu formulieren.

Die Kommunikation zwischen Frontend und Orchestrierungsschicht erfolgt über FastAPI-Schnittstellen. Sobald der Nutzer einen Prompt abschickt, wird innerhalb der Orchestrierungsschicht eine Advanced RAG Pipeline ausgelöst. Der erste Schritt ist eine Routing Funktion, die bestimmt, ob eine Anfrage direkt durch das LLM beantwortet oder zusätzlich internes Wissen benötigt wird. Falls keine interne Wissensbasis erforderlich ist, wird die Anfrage Direkt von einem GPT-4o Modell von OpenAI verarbeitet.

Wird hingegen internes Wissen benötigt, kombiniert die Architektur Pre-Retrieval, Retrieval und Post-Retrieval Schritte (vgl. Kapitel ??), um eine optimierte Suche nach Relevanten Inhalten zu gewährleisten. Darüber hinaus werden zusätzliche Schritte wie Language Identification und Query-Translation verwendet, um eine Mehrsprachige Suche in Deutsch und Englisch zu unterstützen.

Der eigentliche Retrieval Prozess basiert auf einem hybriden Suchansatz. Hierbei werden klassische schlüsselwortbasierte Suchverfahren mit semantischer Vektorsuche kombiniert. Die Suche wird über Azure AI Search realisiert und anschließend durch ein Reranking ergänzt, um besonders relevante Dokumente höher zu priorisieren. Die Ergebnisse verschiedener Suchprozesse werden danach zusammengeführt und anhand eines Similarity Thresholds gefiltert, sodass nur ausreichend relevante Inhalte berücksichtigt werden.

Im Anschluss erfolgt das sogenannte Context Filling, bei dem die gefundenen Dokumente in den Kontext der Anfrage integriert werden. Auf Basis dieses erweiterten Kontextes wird schließlich mithilfe des GPT-4o Modells von OpenAI eine Antwort generiert und an das Frontend zurückgegeben.

Neben der eigentlichen Anfrageverarbeitung beinhaltet die Architektur zusätzlich eine Observability-Schicht. Diese dient der Überwachung und Analyse des Systems. Hierfür kommen unter anderem Langfuse zur Nachverfolgung von Prompts und Modellantworten, Azure Blob Storage zur Datenspeicherung, Snowflake zur Analyse strukturierter Daten sowie Tempo und Grafana zur Visualisierung und Überwachung von Systemmetriken zum Einsatz.

Die Analyse zeigt, dass die bestehende Architektur eine klar strukturierte Trennung zwischen Präsentations- und Orchestrierungsschicht aufweist und durch den Einsatz einer Advanced-RAG-Pipeline eine differenzierte Verarbeitung von Nutzeranfragen ermöglicht. Dabei werden sowohl direkte Modellanfragen als auch erweiterte Retrieval-Prozesse unterstützt.

4.2 Rahmenbedingungen

Die Entwicklung des Artefakts erfolgt unter Berücksichtigung technischer und organisatorischer Rahmenbedingungen, die maßgeblichen Einfluss auf die Gestaltung der Lösung haben.

Zum einen ist das verfügbare Budget begrenzt und bewegt sich im Bereich von ... bis Dies impliziert, dass sowohl Implementierungs- als auch Betriebskosten, insbesondere im Hinblick auf externe Dienste (z. B. Embedding-Modelle oder Vektordatenbanken), sorgfältig abgewogen werden müssen.

Zum anderen ist eine enge Anbindung an die bestehende Systemarchitektur erforderlich. Das zu entwickelnde Artefakt sollte daher möglichst modular gestaltet sein und sich mit geringem Eingriff in die bestehende Infrastruktur integrieren lassen. Eine vollständige Neugestaltung des Systems ist nicht vorgesehen.

Darüber hinaus muss die Lösung mit den vorhandenen Datenstrukturen kompatibel sein und bestehende Persistenzmechanismen sinnvoll erweitern, ohne deren Stabilität oder Performance negativ zu beeinflussen.

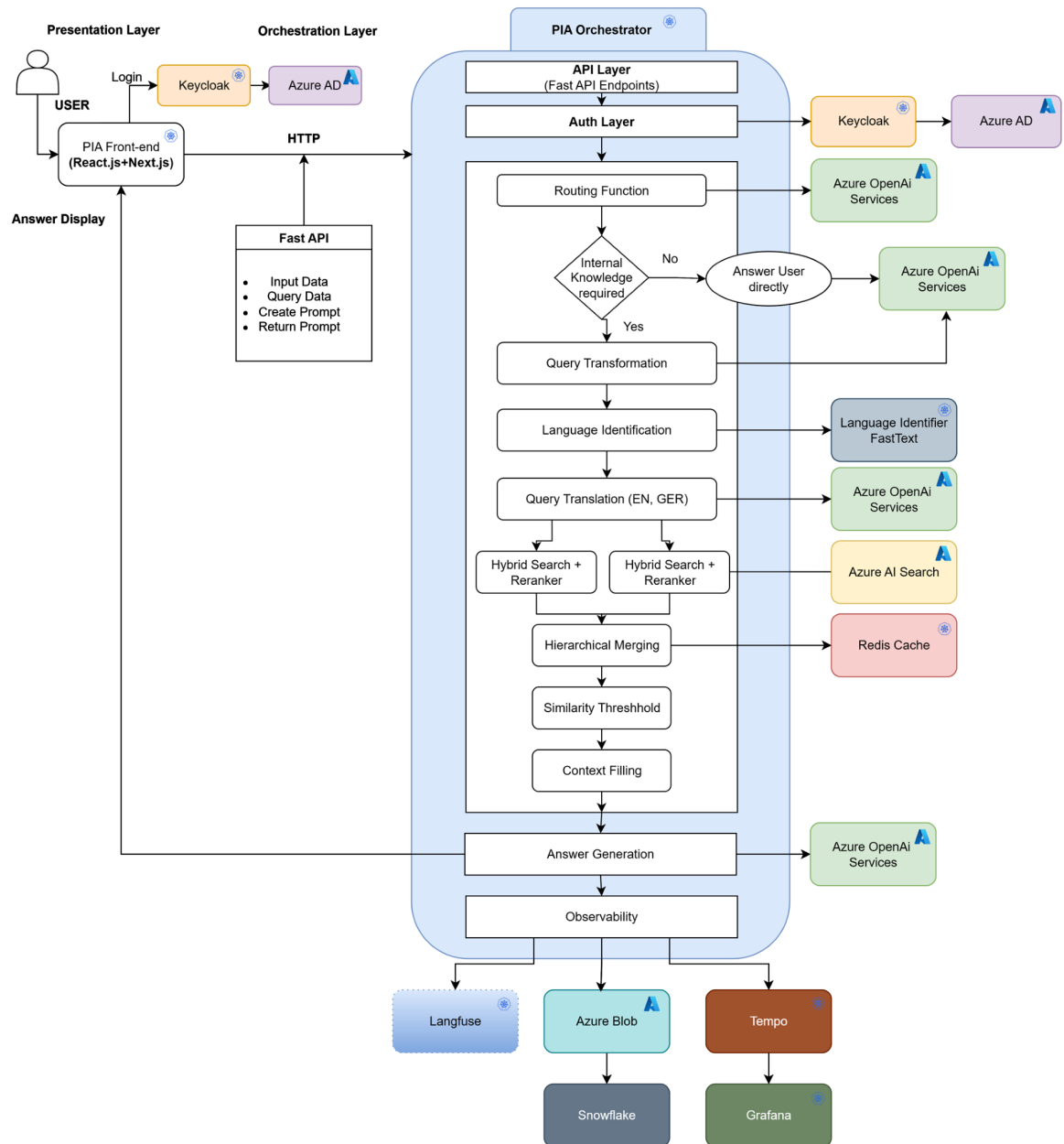
4.3 Ziele

Basierend auf der zuvor durchgeführten Analyse des Problemraums lassen sich folgende zentrale Zielsetzungen für das zu entwickelnde Artefakt ableiten:

- Entwicklung eines Mechanismus zur semantischen Erschließung von Konversationsdaten, insbesondere durch die Transformation von Textinhalten in Vektorrepräsentationen
- Implementierung einer Vektordatenbank zur effizienten Speicherung und Abfrage semantisch ähnlicher Konversationsabschnitte
- Realisierung eines Retrieval-Ansatzes, der relevante historische Informationen kontextabhängig identifiziert und bereitstellt
- Integration der gewonnenen Kontextinformationen in den Anfrageverarbeitungsprozess zur Verbesserung der Antwortqualität
- Ermöglichung der automatisierten Erkennung und Berücksichtigung von Nutzerpräferenzen in zukünftigen Interaktionen
- Sicherstellung einer nahtlosen Integration in die bestehende Systemlandschaft unter Berücksichtigung der definierten Rahmenbedingungen

Ziel des Artefakts ist es somit, die bestehende statische und kontextunabhängige Verarbeitung von Nutzeranfragen durch eine dynamische, kontextbasierte und personalisierte Interaktion zu erweitern.

Abbildung 3: PIA Architektur



5 Evaluation

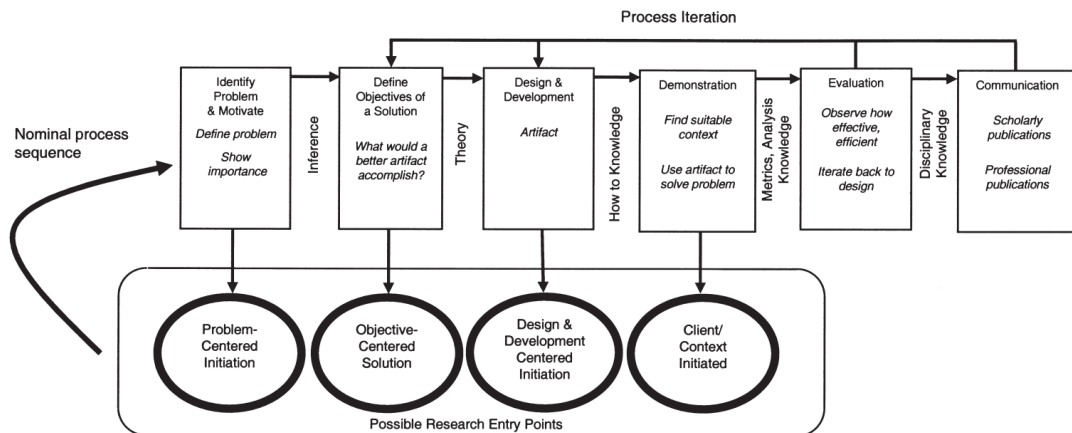
Anhang

Anhangsverzeichnis

Anhang 1:	Alte Schaltlogik	22
Anhang 2:	Testergebnisse	22
Anhang 2.1:	Variablen nach einlesen der XML-Datei	22
Anhang 2.2:	Ergebnisse des Zeitvergleiches	22

Anhang 1 Alte Schaltlogik

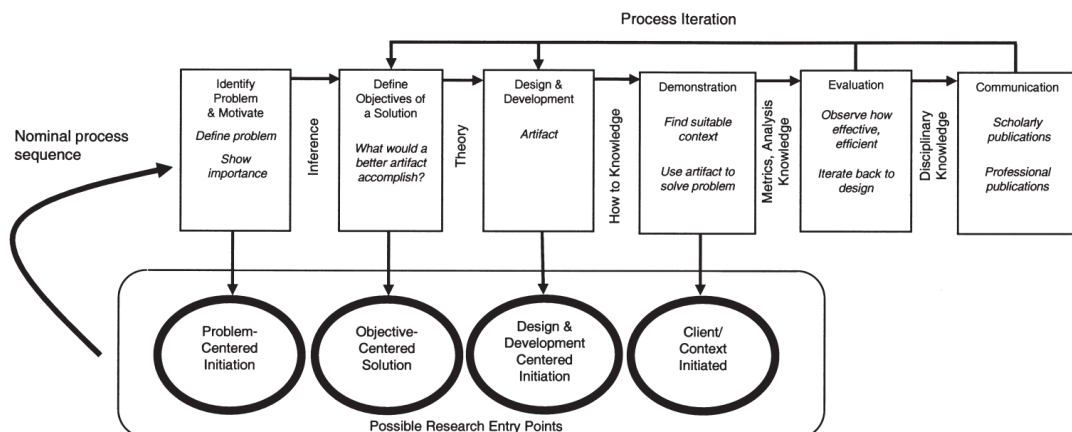
Abbildung 4: Aktuelle Schaltlogik des Funktionsbausteins



Anhang 2 Testergebnisse

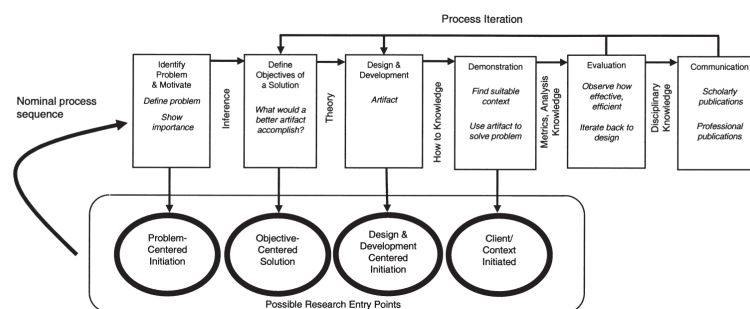
Anhang 2.1 Variablen nach einlesen der XML-Datei

Abbildung 5: Ansicht im Anlagen Ordner



Anhang 2.2 Ergebnisse des Zeitvergleiches

Abbildung 6: Ergebnisse des Zeitvergleiches



Quellenverzeichnis

Monographien

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Guo, Q., Wang, M., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. <https://api.semanticscholar.org/CorpusID:266359151>

Ehrenwörtliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Bachelorthesis selbständig angefertigt habe. Es wurden nur die in der Arbeit ausdrücklich benannten Quellen und Hilfsmittel benutzt. Wörtlich oder sinngemäß übernommenes Gedankengut habe ich als solches kenntlich gemacht. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Paderborn, 8. Juni 2026

Fabio Mirabile